Nama: Sailendra Oriza Yudistira
NIM: 250660221002
Kelas: SI-IIIA

Soal A

Soal A1

1. Class

Class adalah blueprint atau cetak biru untuk membuat objek. Class mendefinisikan atribut dan metode yang dimiliki oleh objek. Contoh penerapan pada kode Python:

```python
class Mobil:
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna
```

2. Object

Object adalah instance dari class. Object memiliki atribut dan metode yang didefinisikan dalam class. Contoh penerapan pada kode Python:

```python
mobil1 = Mobil("Toyota", "Merah")
```

3. Attribute

Atribut adalah variabel yang dimiliki oleh objek. Attribute menyimpan data atau informasi tentang objek tersebut. Contoh penerapan pada kode Python:

```python
print(mobil1.merk)  # Output: Toyota
```

4. Method

Method adalah fungsi yang didefinisikan dalam class dan dapat dipanggil oleh objek. Method digunakan untuk melakukan operasi atau tindakan pada objek. Contoh penerapan pada kode Python:

```python
class Mobil:
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna

    def deskripsi(self):
        return f"Mobil ini adalah {self.merk} berwarna {self.warna}"
```

5. Constructor

Constructor adalah metode khusus dalam class yang digunakan untuk menginisialisasi objek saat dibuat. Constructor biasanya bernama `__init__`. Contoh penerapan pada kode Python:

```python
class Mobil:
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna
```

Soal A2

1. Encapsulation

Encapsulation adalah konsep OOP yang menyembunyikan detail implementasi dari pengguna dan hanya menampilkan antarmuka yang diperlukan. Dalam kode Python, encapsulation dapat diterapkan dengan menggunakan atribut privat (misalnya `self.__nama`) dan property (misalnya `@property`). Atribut privat tidak dapat diakses langsung dari luar class, sehingga melindungi data dari perubahan yang tidak diinginkan. Property memungkinkan akses ke atribut privat melalui metode getter, sehingga pengguna dapat membaca nilai atribut tanpa mengubahnya secara langsung.

```python
class Akun:
    def __init__(self, nama, saldo):
        self.__nama = nama
        self.__saldo = saldo

    @property
    def nama(self):
        return self.__nama

    @property
```

```
    def saldo(self):
        return self.__saldo

    def setor(self, jumlah):
        self.__saldo += jumlah


akun = Akun("Sailendra", 100000)
akun.setor(50000)
print(akun.nama)
print(akun.saldo)
```

2. Inheritance

Inheritance adalah konsep OOP yang memungkinkan sebuah class (subclass) mewarisi atribut dan metode dari class lain (superclass). Dalam kode Python, inheritance diterapkan dengan mendefinisikan subclass yang mewarisi dari superclass menggunakan sintaks `class Subclass(Superclass)`. Subclass dapat menggunakan atribut dan metode dari superclass, serta menambahkan atau mengubah perilaku sesuai kebutuhan. Contohnya, class `Mahasiswa` mewarisi dari class `Pengguna`, sehingga dapat menggunakan atribut `nama` dan metode `tampilkan_peran()` dari class `Pengguna`.

```
class Pengguna:
    def __init__(self, nama):
        self.nama = nama

    def tampilkan_peran(self):
        return f"{self.nama} adalah pengguna."


class Mahasiswa(Pengguna):
    def __init__(self, nama, nim):
        super().__init__(nama)
        self.nim = nim
```

3. Polymorphism

Polymorphism adalah konsep OOP yang memungkinkan objek dari class yang berbeda untuk diakses melalui antarmuka yang sama. Dalam kode Python, polymorphism dapat diterapkan dengan method overriding, di mana subclass dapat mengubah implementasi metode yang diwarisi dari superclass. Contohnya, class `Mahasiswa` dapat mengubah implementasi metode `tampilkan_peran()` dari class `Pengguna`, sehingga ketika metode tersebut dipanggil pada objek

`Mahasiswa`, implementasi yang sesuai dengan subclass akan dijalankan. Polymorphism memungkinkan penggunaan metode yang sama pada objek yang berbeda, sehingga meningkatkan fleksibilitas dan keterbacaan kode.

```
class Hewan:
    def bersuara(self):
        return "Hewan mengeluarkan suara."


class Kucing(Hewan):
    def bersuara(self):
        return "Meong"


class Anjing(Hewan):
    def bersuara(self):
        return "Guk guk"


hewan = [Kucing(), Anjing()]

for h in hewan:
    print(h.bersuara())
```

4. Abstraction

Abstraction adalah konsep OOP yang menyembunyikan detail implementasi dan hanya menampilkan antarmuka yang relevan kepada pengguna. Dalam kode Python, abstraction dapat diterapkan dengan menggunakan class abstrak dan metode abstrak. Class abstrak tidak dapat diinstansiasi secara langsung, tetapi dapat digunakan sebagai blueprint untuk subclass. Metode abstrak didefinisikan dalam class abstrak dan harus diimplementasikan oleh subclass. Contohnya, class `Pengguna` dapat menjadi class abstrak yang mendefinisikan metode abstrak `tampilkan_peran()`, yang harus diimplementasikan oleh subclass seperti `Mahasiswa`. Abstraction membantu menyederhanakan kompleksitas sistem dengan menyembunyikan detail implementasi yang tidak relevan bagi pengguna, sehingga memudahkan pemahaman dan penggunaan class tersebut.

Contoh abstraction sederhana:

```
from abc import ABC, abstractmethod


class Kendaraan(ABC):
    @abstractmethod
```

```
    def bergerak(self):
        pass


class Mobil(Kendaraan):
    def bergerak(self):
        return "Mobil berjalan dengan roda."


mobil = Mobil()
print(mobil.bergerak())
```

Pada contoh tersebut, `Kendaraan` hanya menentukan bahwa setiap kendaraan harus memiliki metode `bergerak()`. Detail cara bergeraknya diserahkan kepada class `Mobil`.

Soal A3

1. `self.__nama` dan `@property` menerapkan konsep **encapsulation**. Awalan `__` membuat atribut menjadi privat melalui name mangling, sehingga tidak dapat diakses langsung dengan nama biasa dari luar class. `@property` menyediakan getter sebagai antarmuka terkontrol untuk membaca nilai atribut tersebut tanpa mengaksesnya secara langsung. Dengan demikian, data terlindungi dan aksesnya dapat diatur melalui method.

2. `class Mahasiswa(Pengguna)` menerapkan konsep **inheritance** (pewarisan). Artinya, `Mahasiswa` mewarisi atribut dan method dari `Pengguna`. `super()` digunakan untuk memanggil implementasi superclass. Pada `super().__init__(nama)`, constructor `Pengguna` dijalankan untuk menginisialisasi `self.nama`, sehingga kode inisialisasi tidak perlu ditulis ulang di subclass.

3. `self.nama` dapat dipanggil di method `tampilkan_peran()` karena atribut tersebut bukan atribut privat. Atribut itu dibuat sebagai `self.nama` di constructor `Pengguna`, sehingga menjadi atribut instance yang dapat diakses oleh method superclass maupun subclass. Atribut privat harus ditulis dengan awalan `__`, misalnya `self.__nama`; atribut tersebut akan mengalami name mangling dan tidak dapat diakses langsung dari subclass menggunakan nama yang sama.

Soal B

Soal B2

1. Identifikasi kesalahan atau kelemahan

1. Atribut `pemilik` dan `saldo` dapat diubah langsung dari luar class, sehingga data rekening tidak terlindungi. Hal ini melanggar prinsip encapsulation.
2. Method `setor()` tidak memvalidasi `jumlah`, sehingga nilai nol atau negatif dapat mengubah saldo secara tidak semestinya.
3. Method `tarik()` tidak memvalidasi jumlah penarikan dan tidak memeriksa saldo. Akibatnya, penarikan negatif dapat dilakukan dan saldo dapat menjadi negatif.

2. Perbaikan kode dengan encapsulation dan validasi

```python
class RekeningBank:
    def __init__(self, pemilik, saldo_awal=0):
        if saldo_awal < 0:
            raise ValueError("Saldo awal tidak boleh negatif.")

        self.__pemilik = pemilik
        self.__saldo = saldo_awal

    @property
    def pemilik(self):
        return self.__pemilik

    @property
    def saldo(self):
        return self.__saldo

    def setor(self, jumlah):
        if jumlah <= 0:
            raise ValueError("Jumlah setor harus positif.")
        self.__saldo += jumlah

    def tarik(self, jumlah):
        if jumlah <= 0:
            raise ValueError("Jumlah tarik harus positif.")
        if jumlah > self.__saldo:
            raise ValueError("Penarikan melebihi saldo.")
        self.__saldo -= jumlah
```

```
    def tampilkan_saldo(self):
        print(f"Saldo {self.pemilik}: Rp{self.saldo}")
```

3. Program utama

```
rekening = RekeningBank("Sailendra", 100000)
rekening.tampilkan_saldo()

rekening.setor(50000)
print("Setor Rp50000 berhasil.")
rekening.tampilkan_saldo()

rekening.tarik(25000)
print("Penarikan Rp25000 berhasil.")
rekening.tampilkan_saldo()

try:
    rekening.tarik(200000)
except ValueError as error:
    print(f"Penarikan ditolak: {error}")

try:
    rekening.tarik(-10000)
except ValueError as error:
    print(f"Penarikan ditolak: {error}")
```